

LAB – WEB SERVICES

Student Details:

Name: Emmanuel Keter

Admission Number: 134976

Unit: Distributed Systems (ICS 4104)

Title: Creation and Consumption of a SOAP Web Service

Group Number: 14

Theory:

A Web Service is a software system that enables communication between different applications over a network using standardized protocols such as HTTP, XML, SOAP, and WSDL. Web services promote interoperability, allowing applications developed using different technologies and programming languages to exchange information seamlessly.

SOAP (Simple Object Access Protocol) is an XML-based messaging protocol used for exchanging structured information between distributed applications. A SOAP Web Service publishes its functionality through a WSDL (Web Services Description Language) document, which clients can use to discover available operations and generate communication stubs automatically.

In this lab, a Calculator Web Service is created and published using Java JAX-WS technology. The service exposes an addition operation that can be invoked remotely by a client application. The client consumes the service through the generated WSDL, demonstrating communication between distributed components.

Aim:

To create a simple SOAP Web Service and develop a distributed application that consumes the web service.

Algorithm:

- Install and configure JDK 8 for JAX-WS support.
- Create a Calculator Web Service class.
- Annotate the service using `@WebService` and `@WebMethod`.
- Create a Publisher class to host the web service endpoint.
- Compile and run the service.
- Verify the generated WSDL in a browser.
- Generate client stubs using the `wsimport` tool.
- Develop a client application to consume the service.
- Invoke the remote addition operation.
- Display the result returned by the web service.

Implementation:

The lab was implemented using Java JAX-WS in a WSL (Ubuntu) environment. The web service was published using the Endpoint.publish() method instead of deploying on GlassFish Server. The generated WSDL was used to create client stubs through the wsimport tool, enabling the distributed client application to invoke the service remotely and receive the computed result.

(a) Software Requirements:

- Windows Subsystem for Linux (WSL)
- Ubuntu
- OpenJDK 8
- Java Compiler (javac)
- JAX-WS (javax.jws, javax.xml.ws)
- wsimport Tool

(b) Installation, File and Folder Creation:

```
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\User>wsl
ekr4tikik@DESKTOP-V8V7511:/mnt/c/Users/User$ cd ~
ekr4tikik@DESKTOP-V8V7511:~$ mkdir -p ~/distributed-systems/web-services
ekr4tikik@DESKTOP-V8V7511:~$ cd ~/distributed-systems/web-services
ekr4tikik@DESKTOP-V8V7511:~/distributed-systems/web-services$ sudo apt update
```

```
ekr4tikik@DESKTOP-V8V7511:~/distributed-systems/web-services$ sudo update-alternatives --config java
There is 1 choice for the alternative java (providing /usr/bin/java).

  Selection    Path                                          Priority  Status
  -----
* 0            /usr/lib/jvm/java-8-openjdk-amd64/jre/bin/java  1081    auto mode
  1            /usr/lib/jvm/java-8-openjdk-amd64/jre/bin/java  1081    manual mode

Press <enter> to keep the current choice[*], or type selection number:
ekr4tikik@DESKTOP-V8V7511:~/distributed-systems/web-services$ sudo update-alternatives --config javac
There is 1 choice for the alternative javac (providing /usr/bin/javac).

  Selection    Path                                          Priority  Status
  -----
* 0            /usr/lib/jvm/java-8-openjdk-amd64/bin/javac    1081    auto mode
  1            /usr/lib/jvm/java-8-openjdk-amd64/bin/javac    1081    manual mode

Press <enter> to keep the current choice[*], or type selection number:
ekr4tikik@DESKTOP-V8V7511:~/distributed-systems/web-services$ java -version
2
javac -version
openjdk version "1.8.0_492"
OpenJDK Runtime Environment (build 1.8.0_492-8u492-ga-us2-0ubuntu1~24.04.1-b09)
OpenJDK 64-Bit Server VM (build 25.492-b09, mixed mode)
2: command not found
javac 1.8.0_492
ekr4tikik@DESKTOP-V8V7511:~/distributed-systems/web-services$ |
```

```
ekr4tikik@DESKTOP-V8V7511:~/distributed-systems/web-services$ mkdir -p com/learn/ws
ekr4tikik@DESKTOP-V8V7511:~/distributed-systems/web-services$ tree
.
├── com
│   └── learn
│       └── ws
└── 4 directories, 0 files
ekr4tikik@DESKTOP-V8V7511:~/distributed-systems/web-services$ |
```

```
ekr4tikik@DESKTOP-V8V7511:~/distributed-systems/web-services$ nano com/learn/ws/Calculator.java
ekr4tikik@DESKTOP-V8V7511:~/distributed-systems/web-services$ |
```

```
GNU nano 7.2 com/learn/ws/Calculator.java *
package com.learn.ws;

import javax.ws.WebService;
import javax.ws.WebMethod;
import javax.ws.WebParam;

@WebService(serviceName = "Calculator")
public class Calculator {

    @WebMethod(operationName = "add")
    public int add(@WebParam(name = "a") int a,
                  @WebParam(name = "b") int b) {

        return a + b;
    }
}

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location  ^U Undo      ^A Set Mark
^X Exit      ^R Read File ^N Replace   ^U Paste     ^J Justify   ^_ Go To Line ^-E Redo     ^-G Copy

ekr4tikik@DESKTOP-V8V7511:~/distributed-systems/web-services$ nano com/learn/ws/CalculatorPublisher.java
ekr4tikik@DESKTOP-V8V7511:~/distributed-systems/web-services$ |
```

```
GNU nano 7.2 com/learn/ws/CalculatorPublisher.java *
package com.learn.ws;

import javax.xml.ws.Endpoint;

public class CalculatorPublisher {

    public static void main(String[] args) {

        Endpoint.publish(
            "http://localhost:9999/ws/calculator",
            new Calculator()
        );

        System.out.println(
            "Service running at http://localhost:9999/ws/calculator?wsdl"
        );
    }
}

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location  ^U Undo      ^A Set Mark
^X Exit      ^R Read File ^_ Replace   ^U Paste     ^J Justify   ^_ Go To Line ^E Redo      ^M Copy
```

(c) Compilation:

```
ekr4tikik@DESKTOP-V8V7511:~/distributed-systems/web-services$ javac com/learn/ws/*.java
ekr4tikik@DESKTOP-V8V7511:~/distributed-systems/web-services$ tree
.
├── com
│   └── learn
│       └── ws
│           ├── Calculator.class
│           ├── Calculator.java
│           ├── CalculatorPublisher.class
│           └── CalculatorPublisher.java
4 directories, 4 files
ekr4tikik@DESKTOP-V8V7511:~/distributed-systems/web-services$ |
```

(d) Execution:

```
ekr4tikik@DESKTOP-V8V7511: X ekr4tikik@DESKTOP-V8V7511: X + v
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\User> wsl
ekr4tikik@DESKTOP-V8V7511:/mnt/c/Users/User$ cd ~
ekr4tikik@DESKTOP-V8V7511:~$ ls
BBIT134976  BBITEC.docx  Backup  distributed-systems  'emmanuel keter.txt'  fourthyearunits.txt  friends.txt  images
ekr4tikik@DESKTOP-V8V7511:~$ cd distributed-sytestems
-bash: cd: distributed-sytestems: No such file or directory
ekr4tikik@DESKTOP-V8V7511:~$ cd distributed-sytestems
-bash: cd: distributed-sytestems: No such file or directory
ekr4tikik@DESKTOP-V8V7511:~$ cd distributed-sytsems
-bash: cd: distributed-sytsems: No such file or directory
ekr4tikik@DESKTOP-V8V7511:~$ cd distributed-systems
ekr4tikik@DESKTOP-V8V7511:~/distributed-systems$ ls
web-services
ekr4tikik@DESKTOP-V8V7511:~/distributed-systems$ cd web-services
ekr4tikik@DESKTOP-V8V7511:~/distributed-systems/web-services$ ls
client  com
ekr4tikik@DESKTOP-V8V7511:~/distributed-systems/web-services$ java com.learn.ws.CalculatorPublisher
Service running at http://localhost:9999/ws/calculator?wsdl
|
```

```
ekr4tikik@DESKTOP-V8V7511:~/distributed-systems/web-services$ mkdir -p client
ekr4tikik@DESKTOP-V8V7511:~/distributed-systems/web-services$ wsimport \
-keep \
-p com.learn.ws.client \
-d client \
http://localhost:9999/ws/calculator?wsdl
parsing WSDL...

Generating code...

Compiling code...

ekr4tikik@DESKTOP-V8V7511:~/distributed-systems/web-services$ curl -s http://localhost:9999/ws/calculator?wsdl | head -20
<?xml version="1.0" encoding="UTF-8"?><!-- Published by JAX-WS RI (http://jax-ws.java.net). RI's version is JAX-WS RI 2.2.9-b130926.1
035 svn-revision#5f6196f2b90e9460065a4c2f4e30e065b245e51e. --><!-- Generated by JAX-WS RI (http://jax-ws.java.net). RI's version is J
AX-WS RI 2.2.9-b130926.1035 svn-revision#5f6196f2b90e9460065a4c2f4e30e065b245e51e. --><definitions xmlns:wsu="http://docs.oasis-open
.org/wss/2004/01/oasis-200401-wss-wssecurity-utility-1.0.xsd" xmlns:wsp="http://www.w3.org/ns/ws-policy" xmlns:wsp1_2="http://schemas
.xmlsoap.org/ws/2004/09/policy" xmlns:wsam="http://www.w3.org/2007/05/addressing/metadata" xmlns:soap="http://schemas.xmlsoap.org/wsdl
/soap/" xmlns:tns="http://ws.learn.com/" xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns="http://schemas.xmlsoap.org/wsdl/" target
Namespace="http://ws.learn.com/" name="Calculator">
<types>
<xsd:schema>
<xsd:import namespace="http://ws.learn.com/" schemaLocation="http://localhost:9999/ws/calculator?xsd=1"></xsd:import>
</xsd:schema>
</types>
<message name="add">
<part name="parameters" element="tns:add"></part>
</message>
<message name="addResponse">
<part name="parameters" element="tns:addResponse"></part>
</message>
<portType name="Calculator">
<operation name="add">
<input wsam:Action="http://ws.learn.com/Calculator/addRequest" message="tns:add"></input>
<output wsam:Action="http://ws.learn.com/Calculator/addResponse" message="tns:addResponse"></output>
</operation>
</portType>
<binding name="CalculatorPortBinding" type="tns:Calculator">
<soap:binding transport="http://schemas.xmlsoap.org/soap/http" style="document"></soap:binding>
ekr4tikik@DESKTOP-V8V7511:~/distributed-systems/web-services$ |
```

```

ekr4tikik@DESKTOP-V8V7511:~/distributed-systems/web-services$ tree
.
├── client
│   ├── com
│   │   ├── learn
│   │   │   └── ws
│   │   │       └── client
│   │   │           ├── Add.class
│   │   │           ├── Add.java
│   │   │           ├── AddResponse.class
│   │   │           ├── AddResponse.java
│   │   │           ├── Calculator.class
│   │   │           ├── Calculator.java
│   │   │           ├── Calculator_Service.class
│   │   │           ├── Calculator_Service.java
│   │   │           ├── ObjectFactory.class
│   │   │           ├── ObjectFactory.java
│   │   │           ├── package-info.class
│   │   │           └── package-info.java
│   └── com
│       ├── learn
│       │   └── ws
│       │       ├── Calculator.class
│       │       ├── Calculator.java
│       │       ├── CalculatorPublisher.class
│       │       └── CalculatorPublisher.java
└── 9 directories, 16 files
ekr4tikik@DESKTOP-V8V7511:~/distributed-systems/web-services$ nano CalculatorClient.java

```

```

GNU nano 7.2 CalculatorClient.java
import com.learn.ws.client.Calculator;
import com.learn.ws.client.Calculator_Service;

public class CalculatorClient {

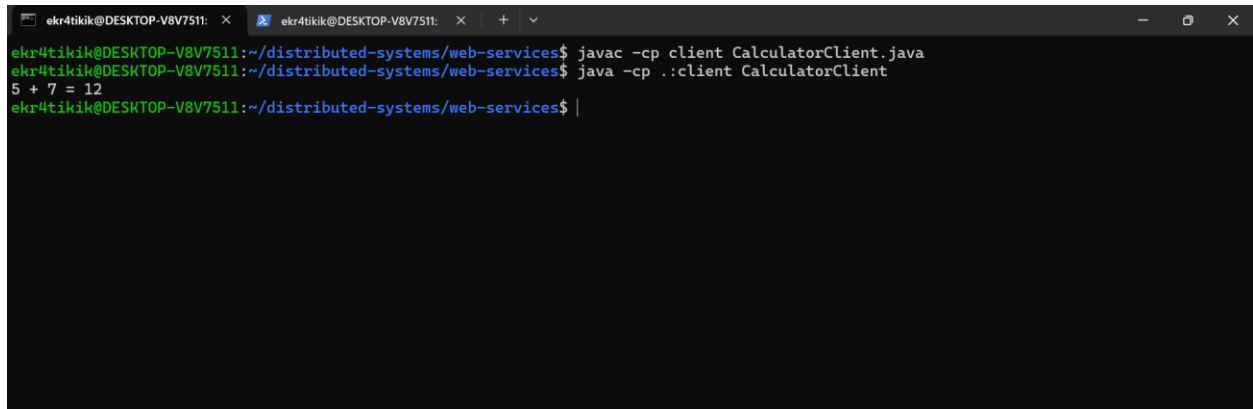
    public static void main(String[] args) {

        Calculator calc =
            new Calculator_Service()
                .getCalculatorPort();

        System.out.println("5 + 7 = " +
            calc.add(5, 7));
    }
}

[ Wrote 15 lines ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line M-E Redo      M-G Copy

```

A terminal window with a dark background and light green text. The window title bar shows two tabs, both labeled 'ekr4tikik@DESKTOP-V8V7511:'. The terminal content shows the user at the directory '~/distributed-systems/web-services' running 'javac -cp client CalculatorClient.java' and then 'java -cp .:client CalculatorClient'. The output of the second command is '5 + 7 = 12'.

```
ekr4tikik@DESKTOP-V8V7511:~/distributed-systems/web-services$ javac -cp client CalculatorClient.java
ekr4tikik@DESKTOP-V8V7511:~/distributed-systems/web-services$ java -cp .:client CalculatorClient
5 + 7 = 12
ekr4tikik@DESKTOP-V8V7511:~/distributed-systems/web-services$ |
```

Conclusion:

The experiment successfully demonstrated the creation and consumption of a SOAP Web Service using Java JAX-WS. The service was published using `Endpoint.publish()` and consumed through client stubs generated using `wsimport`. The implementation achieved the objective of enabling communication between distributed applications through web services.